

Predictive Maintenance for Continuous Production

From reactive downtime to proactive line management. A working reference implementation on a continuous-process plant, with all numbers measured from real telemetry.

Unplanned Downtime Costs **\$33M** Per Year

PER HOUR

\$22K

Entire line output

HOURS PER YEAR

1,500

Across all eight plants

PLUS FREIGHT & SLA

\$10M+

Expedited parts, penalties

One line stopping pulls technicians from other work, forces expedited freight, and often means missing customer commitments.

Why Current Monitoring **Misses the Window**

Plant managers see downtime after it happens. Telemetry exists but is not watched in time.

Traditional RMS vibration alarms miss the critical early phase where intervention is possible.

The Physics

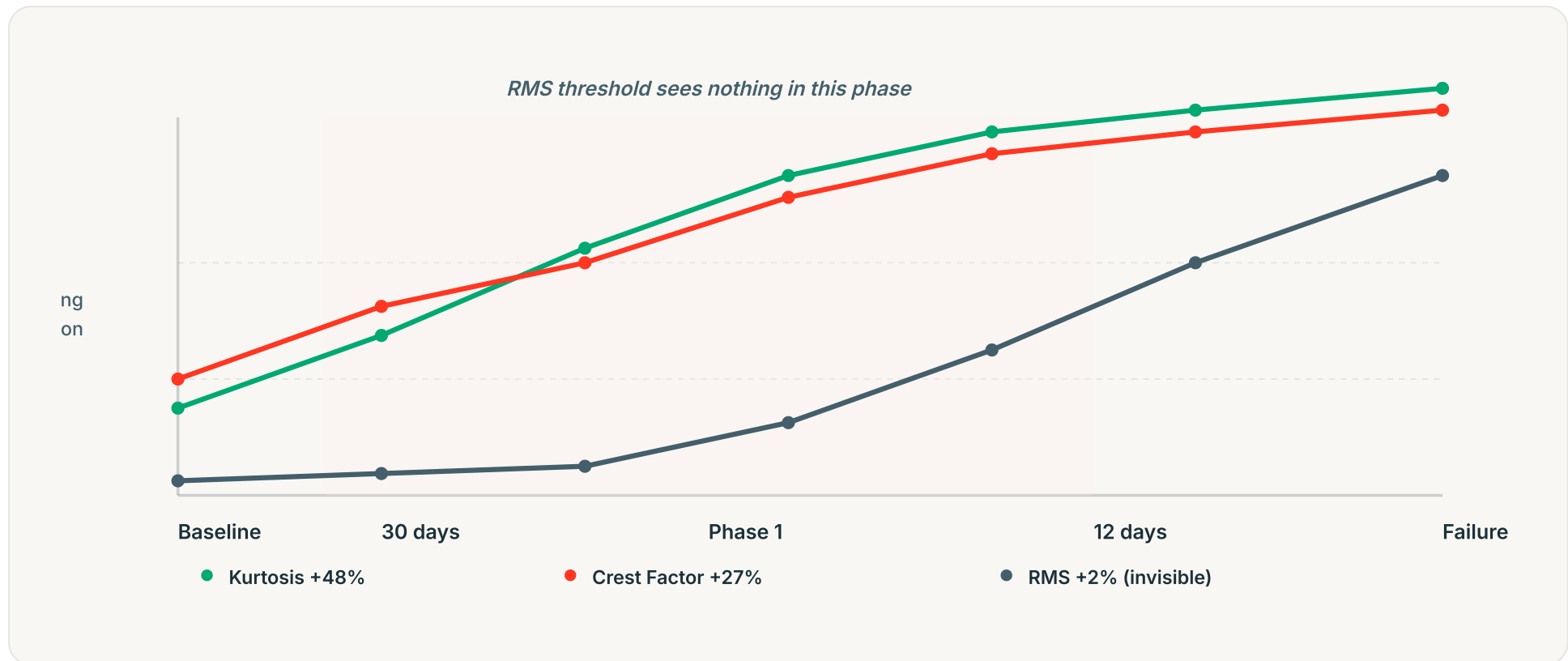
A bearing defect is impulsive. Waveform statistics (crest factor, kurtosis) rise sharply in early degradation while overall energy barely moves.

The Result

An RMS threshold tuned to minimize false positives is blind to the first 18 days. By the time it triggers, options are gone.

What We Measure: The Early Window

Three waveform statistics over 30 days before bearing failure. Phase 1 (30-12 days out) is where the model detects and RMS is blind.



Reference build: bearing degradation over 30 days. Kurtosis and crest factor rise 27-48% in Phase 1 while RMS stays nearly flat. Phase 2 (final 12 days): all three metrics climb sharply.

Reference Implementation on Ironbark Resources

We built and validated the complete system end to end. The numbers below are real measured output from that build. The architecture transfers directly to discrete manufacturing.

- ▶ 151 instrument tags across 25 assets (crushers, screens, conveyors, cyclones, stacker, bins)
- ▶ 21.7 million sensor readings over a single 24-hour window
- ▶ 120 days of historical condition-monitoring data for model training
- ▶ Live mass balance reconciling to 0.01% error on a 2,500-tonne-per-hour feed

This is a continuous-process plant (mining), not a discrete manufacturing line. The architecture transfers directly: same lakehouse structure, same serverless serving layer, same model training loop, same audit trail.

Early Detection: 95.8% vs 14.8%

In the 30-to-12-day window before bearing failure, recall on degrading readings:

Our Model

95.8%

Catches degrading readings in the actionable window. F1 0.988, validated leave-one-asset-out.

RMS Threshold

14.8%

Tuned to minimize false positives. For 3 of 4 failures, it saw nothing until the final 12 days.

The difference is roughly 18 extra days to move a failure from unplanned emergency to scheduled maintenance, with the production plan intact.

Precision: **Zero False Positives**

Batch scoring across all 15 vibration sensors over the 30-day window:

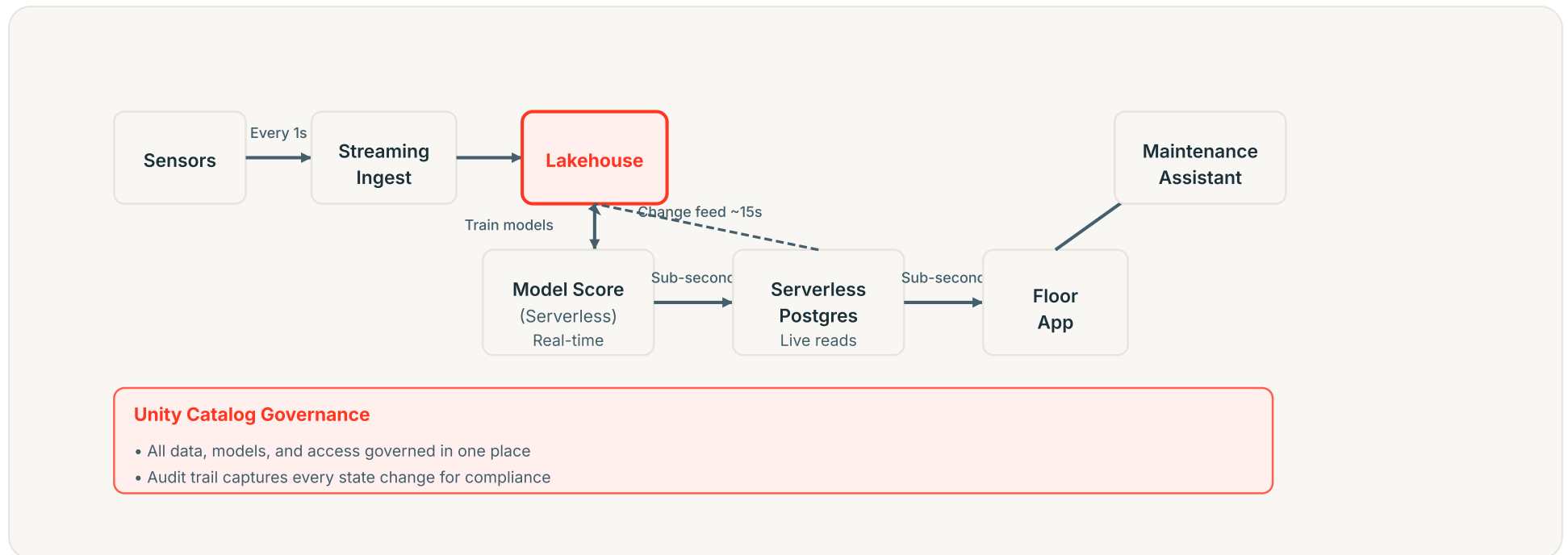
Sensors that actually failed	4 of 15
Model peaked above 0.7 on	exactly 4
Score on the other 11	0.000

A control room that cries wolf gets ignored. The model does not cry wolf. It caught what failed and saw nothing on what did not.

Source: Ironbark Resources reference implementation, batch scoring on reference plant data

How It Reaches the Floor

Telemetry to action, with latency and audit at every layer:



From sensors to floor: streaming ingest, lakehouse, serverless scoring, live Postgres, floor app. Change feed audits every state change back into the lakehouse. Governance spans all layers.

Annika's Question

VP Manufacturing Operations

CARES ABOUT UNPLANNED DOWNTIME REDUCTION

Did a plant manager make a different call this shift?

Yes. In the reference build, the model flagged three bearing defects and a screen failure with 18+ days to spare. Real maintenance was scheduled. Real plant managers had options they did not have before.

That is the job: move failures from after-the-fact emergencies to planned work that fits the production calendar.

The Business Case

Volta's current state and a conservative projection:

Current downtime per year	1,500 hours
---------------------------	--------------------

Cost per hour	\$22,000
---------------	-----------------

Total annual cost	\$33,000,000
-------------------	---------------------

If 10% converts to planned (150 hours)	\$3,300,000
---	--------------------

Achievable with 18 extra days of notice. That is roughly 10 bearing or screen failures caught in the actionable window per year. Your asset mix and failure rates will determine the actual number. We can validate on your historical data in a brief pilot.

Fatima's Question

Director of Manufacturing IT and Finance

CARES ABOUT AI SPEND AT SCALE

If we roll to eight plants, what does it cost?

Cost scales with plants and data, not experiments. The reference environment sits at a 0.5 compute-unit floor and suspends after 300 seconds of idle. Adding a tenth data scientist costs almost nothing.

Eight plants means eight data sources, eight models, eight serving instances. Cost is a function of infrastructure, not friction. We will know the actual spend once we size your data volume.

How Cost Works

Serverless Architecture

- ▶ Compute scales to zero when idle
- ▶ No sustained baseline cost between scoring runs
- ▶ Pricing tied to compute units and data scanned

Discipline Built In

- ▶ Cost is a function of plants and data
- ▶ Not per-experiment or per-model
- ▶ Unity Catalog governs all layers

This is the opposite of a per-experiment pricing model that multiplies out of control. Cost discipline is built into the architecture.

What We Need From You

- ▶ **Access to one plant's data.** Twelve months of telemetry and work orders to validate that the bearing-defect model generalizes to your equipment and to build a plant-specific escalation factor for maintenance decisions.
- ▶ **A technical point person.** Someone who understands your lakehouse setup, infrastructure, and how floor applications connect to your data systems.
- ▶ **Alignment on success.** Early detection in the maintenance backlog? Live production visibility on the floor? Reduced unplanned downtime in the plant manager's metrics? We build for what matters to your business.

The Next Step

We built this reference implementation on a continuous-process plant. Every number comes from that real build. The architecture transfers directly to discrete manufacturing.

Volta's unplanned downtime problem is solvable.

Let's validate it on your data.